
PRODUCT REQUIREMENTS DOCUMENT

Welcome Wave

An AI Onboarding Workflow Platform for Every Employee

Proactive · Verified · Fresh · Role-Scoped · Compounding

Project Title	Welcome Wave
Project Type	Product Development
Theme	Product
Student Names	Vijayabaskar R · Aman Nautiyal
Roll Numbers	251810700080 · 251810700034
Mentor	Sowmiya Mam
Stack / Framework	FastAPI · React · Python · Chroma (Vector DB) · Claude/LLM API · PostgreSQL
Timeline	12 Weeks
Document Version	2.0 — Locked for Build
GitHub	https://github.com/VijayabaskarR-06/welcome-wave

Table of Contents

1. Business Requirements Document (BRD)	6
1.1 Business Context	6
1.2 Business Objectives	6
1.3 Stakeholders	6
1.4 Scope	7
In Scope	7
Out of Scope	7
1.5 Business Requirements	7
1.6 Assumptions	8
1.7 Constraints	9
1.8 Risks	9
2. User Stories	10
Epic 1 — Knowledge Connection & Ingestion	10
Epic 2 — Proactive Onboarding Roadmap	10
Epic 3 — Ask & Retrieve (Grounded Q&A)	11
Epic 4 — Verification & Trust	11
Epic 5 — Staleness & Freshness Detection	11
Epic 6 — Role & Permission Scoping	12
Epic 7 — Manager Analytics & Gaps	12
Epic 8 — Observability	12
Epic 9 — Resilience	12
Epic 10 — Interactive Onboarding Console	13
Epic 11 — Extensibility & Future Integrations	13
3. Technical Requirements Document (TRD)	14
3.1 Project Overview	14
3.2 Core Learning Objectives	14
3.3 Must-Have Requirements	14
3.4 Good-to-Have / Stretch Requirements	14
3.5 Non-Functional Requirements	15
3.6 Technical Requirements Traceability	15
3.7 Technology Rationale	16
Embeddings + Vector Database (Chroma)	16
Claude / LLM API	16

FastAPI (Python)	16
React (Onboarding Console)	16
PostgreSQL	16
Prometheus	7
3.8 Final Deliverables	17
3.9 Project Structure	17
4. High-Level Design (HLD)	19
4.1 System Architecture Overview	19
4.2 Architecture Diagram	19
4.3 Component Responsibilities	19
4.4 Knowledge Flow Layer	20
4.5 End-to-End Flow	20
5. Low-Level Design (LLD)	22
5.1 Repository Structure	22
5.2 Key Interfaces	22
5.3 Verified-Answer Resolution Flow	23
5.4 Role & Permission Scoping	23
5.5 Staleness Dependency Flow	23
5.6 Weekly Implementation Roadmap	24
5.7 Trust Conformance Suite	25
5.8 Observability Design	25
5.9 Design Patterns	25
5.10 Validation & Error Handling	26
6. API Specification	27
6.1 Sources (Ingestion Registry)	27
6.2 Onboarding Roadmap	27
6.3 Ask	27
6.4 Verification	28
6.5 Roles & Permissions	28
6.6 Analytics	28
6.7 Observability	28
6.8 Standard Errors	28
7. Database / Data Design	30
7.1 Core Collections	30
7.2 ER-Style Relationships	31
7.3 Indexes	31

7.4 Data Lifecycle	31
8. UX Requirements — Onboarding Console	33
8.1 Information Architecture	33
8.2 Primary User Flow	33
8.3 Key Screen Requirements	33
8.4 States	33
8.5 Visual Design Direction	34
8.6 Accessibility	34
9. Security Design	35
9.1 Authentication	35
9.2 Authorization	35
9.3 Transport Security	35
9.4 Ingestion-Level Security	35
9.5 Multi-Source Aggregation Security	35
9.6 Observability & Audit	35
9.7 Resilience Against Adversarial Input	35
9.8 Important Product Rule	36
10. Testing Strategy	37
10.1 Test Levels	37
10.2 Sample Test Cases	37
10.3 Trust Conformance Suite	38
10.4 Performance Benchmarking	38
10.5 Resilience / Adversarial Test Matrix	38
10.6 Definition of Done for a Feature	38
11. CI/CD	39
11.1 Pipeline	39
11.2 Branching Strategy	39
11.3 Pull Request Requirements	39
11.4 Quality Gates	39
11.5 Release Process	40
11.6 Rollback	40
12. Observability	41
12.1 Logs	41
12.2 Metrics	41
12.3 Tracing	41

12.4 Dashboards (Onboarding Console)	41
12.5 Alerts	41
12.6 Post-Resilience-Test Review	42
13. Deployment Architecture	43
13.1 Environments	43
13.2 Suggested Deployment	43
13.3 Publishing Targets	43
13.4 Rollback	43
13.5 Configuration Management	43
13.6 Scaling Considerations	43
14. Glossary	44

1. Business Requirements Document (BRD)

1.1 Business Context

Every company, across every department, loses productivity onboarding new employees because knowledge is scattered across documents, code, tickets, CRM notes, and people's heads — and this happens whether the new hire is an engineer, a salesperson, or an HR joiner. General AI tools like ChatGPT, Claude, and NotebookLM can already answer grounded questions over uploaded files, which makes a simple “chatbot over your documents” a commoditized, undefensible product. This project builds Welcome Wave: a production-grade onboarding workflow platform that goes beyond Q&A — it proactively guides every new hire with a role-specific roadmap, lets domain experts curate a verified, trusted knowledge base, and automatically detects when that knowledge goes stale.

1.2 Business Objectives

ID	Objective	Success Signal
BO-01	Give every new hire a proactive, role-specific starting point	Roadmap auto-generates with zero prompt required, across 3+ roles
BO-02	Ground every answer in evidence, never presented as unverified fact	100% of AI-generated answers carry a source citation and a draft/verified label
BO-03	Let domain experts curate what's actually true	Verify workflow turns a draft into a canonical, owner-stamped answer
BO-04	Keep the knowledge base honest as source material changes	Source-change events flag all dependent verified answers for re-verification
BO-05	Serve any department, not just engineering	Same engine correctly scoped and demoed across 3+ distinct roles
BO-06	Prove this isn't a thin AI wrapper	System explains and demonstrates its differentiation from ChatGPT/Claude/NotebookLM live

1.3 Stakeholders

Stakeholder	Interest
Student engineering team (Vijayabaskar R, Aman Nautiyal)	Build and ship a production-grade onboarding workflow platform
New hires across departments (engineering, sales, HR, support, ops)	A fast, trustworthy way to get productive without interrupting colleagues

Stakeholder	Interest
Domain experts / team leads	A way to bless correct answers once and stop repeating themselves
People/HR & engineering managers	Visibility into ramp time and documentation gaps across teams
Evaluators / reviewers	A working system, clear documentation, and a recorded demo

1.4 Scope

In Scope

- Read-only knowledge source connection (document upload + optional GitHub repo)
- Role- and permission-tagged ingestion and chunking
- Semantic retrieval with role-scoped access
- Grounded answer generation with evidence citations
- AI-draft vs Verified labeling on every answer
- Verification workflow (domain expert to canonical answer)
- Staleness detection (source change to re-verification flag)
- Auto-generated, role-specific onboarding roadmap
- Manager analytics dashboard (recurring/unanswered questions, verified-coverage)
- Interactive onboarding console covering roadmap, ask, verify queue, and dashboard
- Observability via Prometheus-compatible metrics
- Recorded demo and architecture documentation

Out of Scope

- Building a new foundation LLM or embedding model
- Live enterprise connectors (Confluence, Slack, Salesforce, Zendesk, HRIS) — document upload/GitHub only for MVP
- Multi-tenant billing, organization account management, or commercial SaaS packaging
- Mobile client implementations
- Self-hosted/private-VPC deployment (roadmap only)

1.5 Business Requirements

ID	Requirement	Priority
BR-001	System shall ingest a connected knowledge source with role/permission metadata	Must-Have

ID	Requirement	Priority
BR-002	System shall generate an auto role-specific onboarding roadmap on hire creation	Must-Have
BR-003	System shall answer a question with a grounded, evidence-cited response	Must-Have
BR-004	System shall label every answer as AI-draft or Verified	Must-Have
BR-005	System shall let a domain expert verify a draft into a canonical answer	Must-Have
BR-006	System shall serve verified answers ahead of newly generated drafts	Must-Have
BR-007	System shall detect source changes and flag dependent verified answers stale	Good-to-Have
BR-008	System shall scope retrieval and roadmap content by role	Must-Have
BR-009	System shall expose Prometheus-compatible metrics	Good-to-Have
BR-010	System shall provide a manager dashboard of recurring gaps and verified-coverage	Good-to-Have
BR-011	System shall never write back to a connected knowledge source (read-only)	Must-Have
BR-012	System shall survive malformed input and concurrent load without crashing	Good-to-Have

1.6 Assumptions

- The team has access to an embeddings API, a vector database, and an LLM API (Claude) for grounded generation.
- The project runs over a 12-week, production-grade build timeline with a team of two.
- No real employee PII flows through the system during development — all seed knowledge and test data are synthetic or already-public sample material.

1.7 Constraints

- Timeline: 12 weeks, phased delivery.
- Team size: 2 engineers.
- Compatibility: Ingestion must work with common company knowledge formats (PDF, markdown, docx) plus GitHub without bespoke configuration per source.
- Positioning constraint: every feature must defend against “why not just use ChatGPT” — commoditized Q&A alone does not satisfy the product’s business objective.

1.8 Risks

Risk	Impact	Mitigation
Scope creep toward building every live connector	Delays core differentiators	Lock MVP to document upload + GitHub only; connectors are roadmap
Answer hallucination on incomplete context	Erodes trust in verified system	Strict “answer only from retrieved context” prompting + explicit “not found” fallback
Staleness-dependency graph becomes inconsistent	Wrong answers marked trusted	Every verified answer stores explicit source-chunk links, tested via a trust conformance suite
Demo looks like a thin AI wrapper	Fails to differentiate from ChatGPT/Claude	Demo always leads with roadmap, verify, and staleness flag; chat is shown last

2. User Stories

Personas used below: **New Hire** (any role), **Domain Expert** (verifies answers in their area), **Manager / Admin** (configures roles, reviews analytics), and **Future Integrator** (a later engineer adding a new knowledge connector).

Epic 1 — Knowledge Connection & Ingestion

US-001 As an Admin, I want to connect a knowledge source with read-only access, so that Welcome Wave can index it without any risk of modifying company systems.

Acceptance Criteria:

- Admin can upload documents (PDF, markdown, docx) or connect one GitHub repo
- No write access is ever requested or used
- Source connects without needing ongoing manual re-upload

US-002 As an Admin, I want to scope exactly what gets indexed, so that sensitive or irrelevant content is excluded.

Acceptance Criteria:

- Admin can include/exclude specific files or folders before indexing
- Excluded content is never embedded or retrievable

Epic 2 — Proactive Onboarding Roadmap

US-003 As a New Hire, I want a role-specific roadmap the moment I'm added, so that I have a clear starting point instead of a blank chat box.

Acceptance Criteria:

- Roadmap auto-generates on hire creation, no prompt required
- Includes an overview, "read this first" items, and a suggested first task
- Content derives from the role's tagged knowledge, not generic text

US-004 As a New Hire, I want the roadmap to be the first thing I see, so that I'm guided rather than left to figure out what to ask.

Acceptance Criteria:

- Roadmap is the default landing view for a new hire, not the chat screen

Epic 3 — Ask & Retrieve (Grounded Q&A)

US-005 As any Employee, I want to ask a question in plain English and get a grounded answer, so that I don't have to search manually.

Acceptance Criteria:

- Question is embedded and matched against role-permitted content only
- Answer is generated strictly from retrieved context
- System states so rather than guessing if no relevant content exists

US-006 As any Employee, I want to see exactly where an answer came from, so that I can verify it myself.

Acceptance Criteria:

- Every answer displays clickable evidence (document section or file/PR reference)

Epic 4 — Verification & Trust

US-007 As a Domain Expert, I want to verify a correct AI draft, so that it becomes the trusted answer for everyone after me.

Acceptance Criteria:

- Expert can click Verify (optionally edit first) on any draft answer
- Verified answer is stamped with verifier name and timestamp
- Stored as canonical, separate from the draft

US-008 As any Employee, I want to be served the verified answer first if one exists, so that I get the trusted version instead of a fresh AI guess.

Acceptance Criteria:

- Verified store is checked before generating a new draft
- Verified answers are returned instantly, without a new LLM call

Epic 5 — Staleness & Freshness Detection

US-009 As a Domain Expert, I want to be notified when a source I verified from has changed, so that I don't leave outdated information marked as trusted.

Acceptance Criteria:

- Each verified answer stores a dependency link to its source chunk(s)
- Source change flags all dependent verified answers "needs re-verification"

- Flagged answers are visually downgraded until re-confirmed

US-010 As a Domain Expert, I want a single queue of everything needing re-verification, so that I can clear the backlog efficiently.

Acceptance Criteria:

- A “needs re-verification” queue is filterable by domain/owner

Epic 6 — Role & Permission Scoping

US-011 As an Admin, I want each role to see only the knowledge and roadmap permitted for it, so that sensitive information isn't exposed to the wrong role.

Acceptance Criteria:

- Two roles with different permissions receive different retrieval results and roadmaps
- A role cannot retrieve content outside its scope, even via a direct question

Epic 7 — Manager Analytics & Gaps

US-012 As a Manager, I want to see which questions keep coming up unanswered, so that I know where documentation is weakest.

Acceptance Criteria:

- Dashboard lists top recurring/unanswered questions per role
- Verified-coverage percentage is shown per role

Epic 8 — Observability

US-013 As a Platform Operator, I want latency and usage metrics for ingestion and retrieval, so that I can monitor system health.

Acceptance Criteria:

- /metrics endpoint exposes Prometheus-compatible counters/histograms
- Metrics are labelled by role and operation type

Epic 9 — Resilience

US-014 As a Platform Operator, I want the system to degrade gracefully under malformed input or high concurrency, so that a single bad request doesn't take down the service.

Acceptance Criteria:

- Malformed ingestion input returns a typed error, not a crash
- Concurrent question load does not corrupt the verified store

Epic 10 — Interactive Onboarding Console

US-015 As a New Hire or Evaluator, I want a web console covering roadmap, ask, verify queue, and dashboard, so that I can explore the whole system without needing separate tools.

Acceptance Criteria:

- Console shows a role switcher, roadmap view, and ask/evidence view
- Domain experts see the verify queue; managers see the analytics dashboard

Epic 11 — Extensibility & Future Integrations

US-016 As a future Integrator, I want the ingestion layer built as a pluggable connector interface, so that new sources (Confluence, Slack, HRIS) can be added without redesigning the core engine.

Acceptance Criteria:

- Ingestion connectors implement a common interface (fetch, chunk, tag)
- Adding a new connector type requires no change to retrieval, verification, or staleness logic

3. Technical Requirements Document (TRD)

3.1 Project Overview

This document covers the full design and implementation plan for Welcome Wave, a production-grade AI onboarding workflow platform. The goal is to go beyond a chatbot wrapper — every layer from knowledge ingestion to verification to staleness detection is built to production standards and demoed across multiple real-world departments.

3.2 Core Learning Objectives

Domain	What We Build Towards
AI / LLM Engineering	Designing retrieval-augmented generation that is strictly grounded and honest about uncertainty, not just fluent
Product & Workflow Design	Building a proactive workflow around an AI capability, rather than a reactive chatbot
Distributed Data Design	Role/permission-scoped indexing, dependency-linked staleness detection, verified knowledge as a first-class datastore
Trust & Governance Engineering	Ownership, audit trail, and re-verification workflows for AI-generated content
Full-Stack Product Delivery	Shipping a usable, demoable console end-to-end across ingestion, retrieval, verification, and analytics

3.3 Must-Have Requirements

- Read-only knowledge ingestion with role/permission tagging
- Role-scoped semantic retrieval with grounded, cited answer generation
- AI-draft vs Verified labeling on every answer
- Verification workflow producing canonical, owner-stamped answers
- Auto-generated, role-specific onboarding roadmap
- Demoable across 3+ configured roles with zero manual reconfiguration mid-demo

3.4 Good-to-Have / Stretch Requirements

- Staleness detection with automatic re-verification flagging
- Manager analytics dashboard (recurring gaps, verified-coverage)
- Prometheus-compatible observability

- Knowledge Risk Score (bus-factor per domain)
- Exit Capture flow for departing domain experts
- Pluggable connector interface for future live sources (Confluence, Slack, HRIS)

3.5 Non-Functional Requirements

Category	Target	Notes
Groundedness	100% of answers sourced only from retrieved context	No answer generated from general model knowledge alone
Trust labeling	100% of answers labeled draft or verified	Never presented ambiguously
Role Scoping	Zero cross-role leakage	A role cannot retrieve content outside its permission scope
Freshness	100% of dependent verified answers flagged on source change	Verified by dependency-link tests
Latency	Grounded answer returned within a few seconds	Acceptable for a live, multi-role demo
Demo Reliability	Zero failures across full script	Rehearsed across all configured roles
Documentation	Architecture doc + recorded demo	Written and video deliverable for the full system

3.6 Technical Requirements Traceability

ID	Requirement	Maps to
TR-001	Read-only ingestion connector interface	BR-001
TR-002	Role/permission metadata on every chunk	BR-001, BR-008
TR-003	Role-specific roadmap generator	BR-002
TR-004	Role-filtered semantic retrieval	BR-003, BR-008
TR-005	Grounded answer generation with evidence citation	BR-003
TR-006	AI-draft / Verified labeling	BR-004
TR-007	Verification workflow (draft to canonical)	BR-005
TR-008	Verified-first retrieval priority	BR-006

ID	Requirement	Maps to
TR-009	Source-to-answer dependency links + staleness flag	BR-007
TR-010	Prometheus metrics exporter	BR-009
TR-011	Manager analytics dashboard	BR-010
TR-012	Read-only enforcement at connector level	BR-011
TR-013	Malformed-input and load resilience handling	BR-012

3.7 Technology Rationale

Embeddings + Vector Database (Chroma)

Lightweight, self-hostable semantic search with no heavy infrastructure needed for an MVP-scale knowledge base, while still supporting role-filtered retrieval via metadata.

Claude / LLM API

Chosen for answer generation because strict context-grounding and refusal-to-guess behavior are easier to enforce with clear system instructions — central to the product's trust story.

FastAPI (Python)

Fast to build ingestion, retrieval, and verification endpoints with typed request/response models, and pairs naturally with the Python-first AI/embeddings ecosystem.

React (Onboarding Console)

Chosen for the console (roadmap, ask, verify queue, dashboard) where component reuse and role-based view switching are natural fits.

PostgreSQL

Stores the verified-answer store, roles/permissions, and dependency links as durable, queryable, relational data — distinct from the ephemeral-feeling vector index.

Prometheus

Industry-standard metrics format, letting observability plug into any standard monitoring stack without custom tooling.

3.8 Final Deliverables

1. A working Welcome Wave instance demoable across 3+ configured roles
2. A written architecture document covering all design decisions and trade-offs
3. A recorded demo video walking through the full system, including the staleness-flag moment
4. A role-based onboarding console (roadmap, ask, verify queue, manager dashboard)
5. Prometheus-compatible metrics and a basic observability view
6. A documented, pluggable connector interface ready for future live sources

3.9 Project Structure

```
welcome-wave/  
  .github/  
    workflows/  
      ci.yml  
  packages/  
    ingestion/          # Knowledge connectors  
      src/  
        connectors/     # document-upload, github (pluggable interface)  
        chunker/        # role/permission-aware chunking  
        embeddings/     # embedding generation  
        index.ts  
    retrieval/          # Role-scoped semantic search + answer generation  
      src/  
        search/  
        generation/     # grounded LLM answer generation  
        evidence/        # citation resolution  
        index.ts  
    verification/       # Verify workflow + canonical store  
      src/  
        store/           # verified-answer datastore  
        workflow/        # draft to canonical logic  
        staleness/       # dependency links + flagging  
        index.ts  
    roadmap/            # Role-based roadmap generator  
      src/  
        generator/  
        index.ts  
    console/            # React onboarding console  
      src/  
        roadmap/  
        ask/  
        verifyQueue/  
        dashboard/  
        App.tsx  
  tests/  
    unit/  
    integration/  
    resilience/  
  docs/  
    architecture.md
```

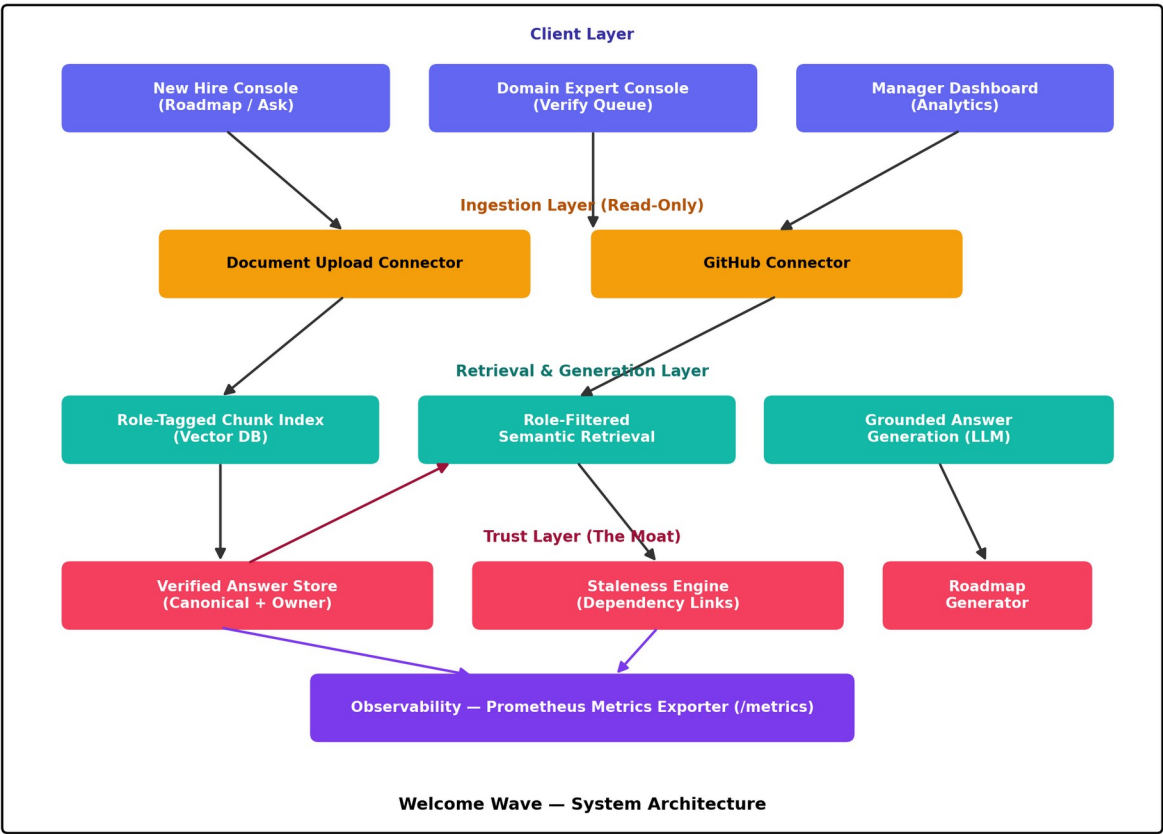
```
api-reference.md
scripts/
  setup-dev-env.sh
.env.example
docker-compose.yml
package.json
pyproject.toml
README.md
```

4. High-Level Design (HLD)

4.1 System Architecture Overview

Welcome Wave has three primary layers: an **Ingestion layer** that connects and indexes company knowledge sources (read-only), a **Retrieval & Generation layer** that answers questions with role-scoped, grounded, evidence-cited responses, and a **Trust layer** that turns AI drafts into a verified, staleness-aware knowledge asset. A Roadmap generator and a Manager analytics view sit on top, both driven by the same underlying role-tagged index and verified store.

4.2 Architecture Diagram



4.3 Component Responsibilities

Component	Technology	Responsibility
Ingestion Connectors	Python (FastAPI)	Pull source content read-only; tag role/permission; hand off to chunker

Component	Technology	Responsibility
Chunker + Embeddings	Python + embeddings API	Split content into role-tagged chunks; generate vector embeddings
Vector Index	Chroma	Store embeddings for fast role-filtered semantic search
Retrieval & Generation	FastAPI + LLM API	Role-scoped search; strictly grounded answer generation with evidence
Verified Answer Store	PostgreSQL	Canonical answers, owners, timestamps, source-dependency links
Staleness Engine	Python service	Detects source changes; flags dependent verified answers
Roadmap Generator	Python service	Builds role-specific onboarding path from tagged knowledge
Onboarding Console	React	Roadmap, Ask, Verify Queue, and Manager Dashboard views
Observability	Prometheus + custom exporter	Latency, usage, and error metrics across ingestion and retrieval

4.4 Knowledge Flow Layer

Two knowledge motions are supported: **seed ingestion** (an admin connects a source once, read-only) and **organic growth** (a domain expert verifying a draft answer effectively adds new trusted content, without anyone “documenting” anything separately). Both flow into the same role-tagged index and verified store.

4.5 End-to-End Flow

Step	Who	What Happens
1	Admin	Connects a knowledge source (read-only); scopes what's indexed
2	System	Ingests, chunks, embeds, and tags content with source/role/permission
3	Admin	Assigns a new hire a role
4	System	Auto-generates the role-specific onboarding roadmap
5	Employee	Opens Welcome Wave; lands on

Step	Who	What Happens
		the roadmap, not a blank chat
6	Employee	Asks a question
7	System	Checks the verified store first; if found, serves it labeled “Verified”
8	System	If not found, retrieves role-scoped chunks and generates a grounded draft, labeled “AI draft — unverified”
9	Domain Expert	Reviews and verifies useful drafts into canonical answers
10	System	On a source update, flags all dependent verified answers “needs re-verification”
11	Manager	Reviews the dashboard for recurring gaps and re-verification backlog
12	Observability	Every step emits metrics — latency, retrieval hits, verification events — to Prometheus

5. Low-Level Design (LLD)

5.1 Repository Structure

See Section 3.9 for the full repository layout. The LLD below focuses on the ingestion, retrieval, and verification packages, which carry the core protocol logic.

5.2 Key Interfaces

IngestionConnector

```
interface IngestionConnector {
  connect(config: SourceConfig): Promise<void>
  fetch(): Promise<RawDocument[]>
  // read-only -- no write/update/delete method exists on this interface
}
```

Chunker

```
interface Chunker {
  chunk(doc: RawDocument, role: string, permission: string): Chunk[]
}
```

RetrievalEngine

```
interface RetrievalEngine {
  search(query: string, role: string, topK: number): Promise<Chunk[]>
  generateAnswer(query: string, chunks: Chunk[]): Promise<DraftAnswer>
}
```

VerificationStore

```
interface VerificationStore {
  verify(draftId: string, expertId: string): Promise<VerifiedAnswer>
  getVerified(query: string, role: string): Promise<VerifiedAnswer | null>
  flagStale(sourceId: string): Promise<VerifiedAnswer[]>
}
```

RoadmapGenerator

```
interface RoadmapGenerator {
  generate(role: string): Promise<Roadmap>
}
```

5.3 Verified-Answer Resolution Flow

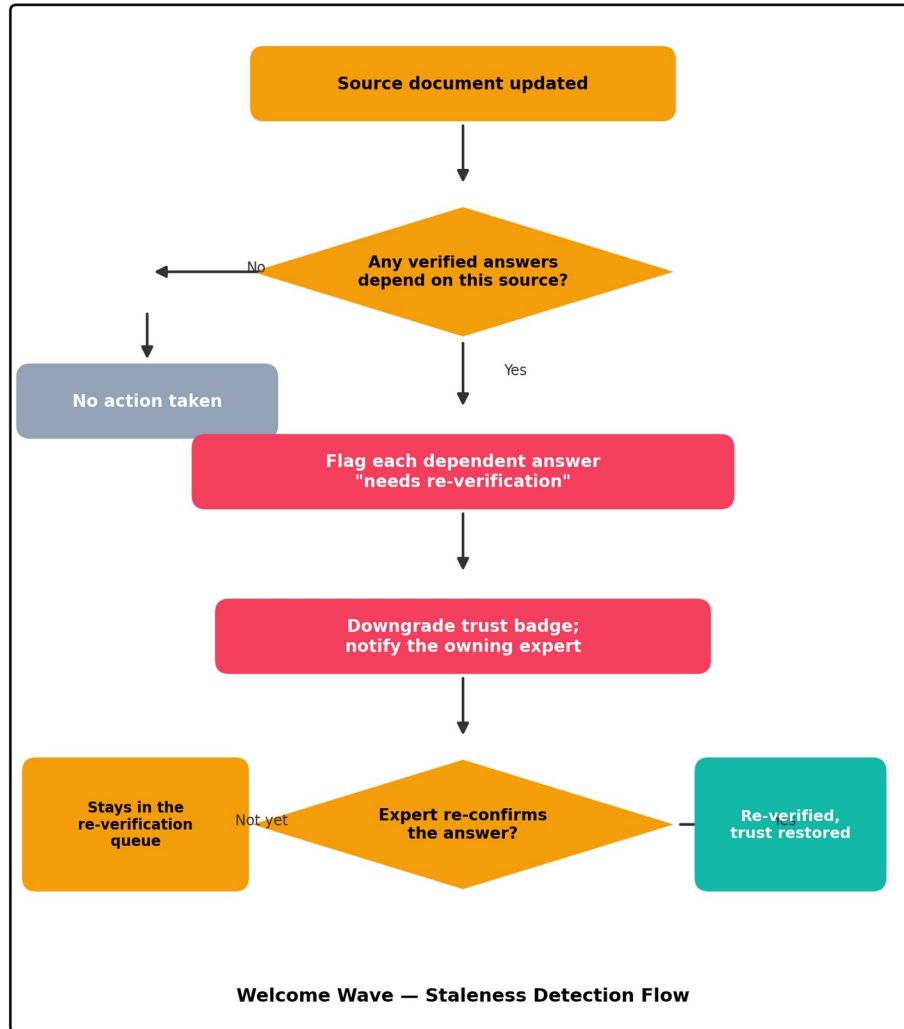
On each question, the system first checks `VerificationStore.getVerified()`. If a match exists, it is returned immediately, labeled trusted. If not, `RetrievalEngine.search()` and `generateAnswer()` produce a labeled draft. This mirrors a priority-negotiation flow, but resolves to “trusted vs draft” rather than a protocol version.

5.4 Role & Permission Scoping

Each chunk and each roadmap template carries a role tag. `RetrievalEngine.search()` filters at the vector-index query level, never post-filtering after retrieval, so content outside a role’s scope is never returned even indirectly.

5.5 Staleness Dependency Flow

Every `VerifiedAnswer` stores an array of source chunk IDs it was built from. When a source is re-ingested (changed), the Staleness Engine looks up every `VerifiedAnswer` referencing an affected chunk ID and calls `flagStale()`, downgrading its trust label until an expert re-confirms it.



5.6 Weekly Implementation Roadmap

Phase	Weeks	Focus	Deliverable
Foundation	W1	Repo setup, environment config, role/permission data model	Working dev environment; basic project structure
Ingestion	W2	Document-upload + GitHub connectors, chunking with metadata	Source connects; role-tagged chunks produced
Retrieval Core	W3-W4	Embeddings, vector index, role-filtered search, grounded generation	End-to-end: ask a question, get a grounded, cited draft
Roadmap	W5	Role-based roadmap	New hire sees an auto-

Phase	Weeks	Focus	Deliverable
		generator	generated roadmap on assignment
Verification	W6–W7	Verify workflow, canonical store, verified-first retrieval	Expert can verify a draft; it's served trusted afterward
Staleness	W8	Source-dependency links, re-index detection, flagging	Source change correctly flags dependent verified answers
Analytics	W9	Manager dashboard (gaps, verified-coverage)	Dashboard shows real data across configured roles
Console Polish	W10	Full onboarding console UI across all views	Roadmap / Ask / Verify Queue / Dashboard all usable end-to-end
Hardening	W11	Resilience testing, malformed-input handling, load check	System survives bad input and concurrent use without crashing
Demo	W12	Architecture documentation, recorded demo video	Full deliverable set complete and rehearsed

5.7 Trust Conformance Suite

A reusable internal test suite validates: role-scoped retrieval never leaks cross-role content, every verified answer resolves before a new draft is generated, and every source change correctly flags all and only its dependent verified answers — the equivalent of a conformance suite, scoped to this product's trust guarantees.

5.8 Observability Design

Every ingestion and retrieval operation is instrumented. The console's manager dashboard pulls these metrics live for the analytics view. (Full metric list in Section 12.)

5.9 Design Patterns

- Repository pattern for the verified-answer and role/permission stores
- Strategy pattern for interchangeable ingestion connectors (document-upload vs GitHub vs future live connectors)
- Observer-style dependency tracking for staleness propagation
- Dependency injection across ingestion, retrieval, and verification packages for testability

5.10 Validation & Error Handling

- Every inbound ingestion request is validated against its source-config schema before connecting
- Every generated answer is checked for context-groundedness before being labeled and returned
- All errors are typed (IngestionError, RetrievalError, VerificationError, StalenessError) and carry a stable error code plus request/session ID
- User-facing errors never expose internal stack traces

6. API Specification

Base URL: /api/v1

6.1 Sources (Ingestion Registry)

POST /sources — connect a new knowledge source (read-only)

```
{
  "name": "sales-playbook",
  "type": "document-upload",
  "role": "sales",
  "files": ["pricing.pdf"]
}
```

GET /sources — list connected sources and index status

DELETE /sources/:id — disconnect a source (index only, never touches the original)

6.2 Onboarding Roadmap

GET /roadmap/:role — returns the auto-generated roadmap for a role

```
{
  "role": "sales",
  "overview": "...",
  "readFirst": ["pricing.pdf#section-2"],
  "firstTask": "Shadow a live demo call"
}
```

6.3 Ask

POST /ask

```
{ "question": "What's our discount policy?", "role": "sales" }
```

Response:

```
{
  "answer": "...",
  "status": "verified",
  "evidence": [{ "source": "pricing.pdf", "section": "2.1" }],
  "verifiedBy": "priya@company.com",
  "verifiedAt": "2026-09-02T10:00:00Z"
}
```

6.4 Verification

POST /answers/:id/verify

```
{ "expertId": "priya@company.com", "editedAnswer": "optional override text" }
```

Response:

```
{ "id": "...", "status": "verified", "verifiedBy": "...", "verifiedAt": "..." }
```

GET /answers/needs-reverification — returns the staleness queue

6.5 Roles & Permissions

POST /roles

```
{ "name": "sales", "permittedSources": ["sales-playbook"] }
```

GET /roles — list configured roles

6.6 Analytics

GET /analytics/gaps — top recurring/unanswered questions per role

GET /analytics/coverage — verified-coverage percentage per role

6.7 Observability

GET /metrics — Prometheus-formatted metrics

GET /health — liveness/readiness probe

6.8 Standard Errors

```
{
  "error": {
    "code": "VERIFICATION_ERROR",
    "message": "The answer could not be verified.",
    "requestId": "req_123"
  }
}
```

Status	Meaning
400	Validation error (bad parameters /

Status	Meaning
	malformed request)
401	Authentication failure (missing/invalid token)
403	Role-scope violation (content outside permitted role)
404	Missing resource / source / answer
409	Conflict (e.g. duplicate source registration)
422	Semantic validation failure (well-formed but invalid input)
500 / 503	Server or downstream dependency failure

Error Code	Typical Cause
INGESTION_ERROR	Source content could not be read or chunked
RETRIEVAL_ERROR	Vector index search failed
VERIFICATION_ERROR	Draft could not be promoted to canonical
STALENESS_ERROR	Dependency-link resolution failed
ROLE_SCOPE_ERROR	Requested content is outside the caller's role scope

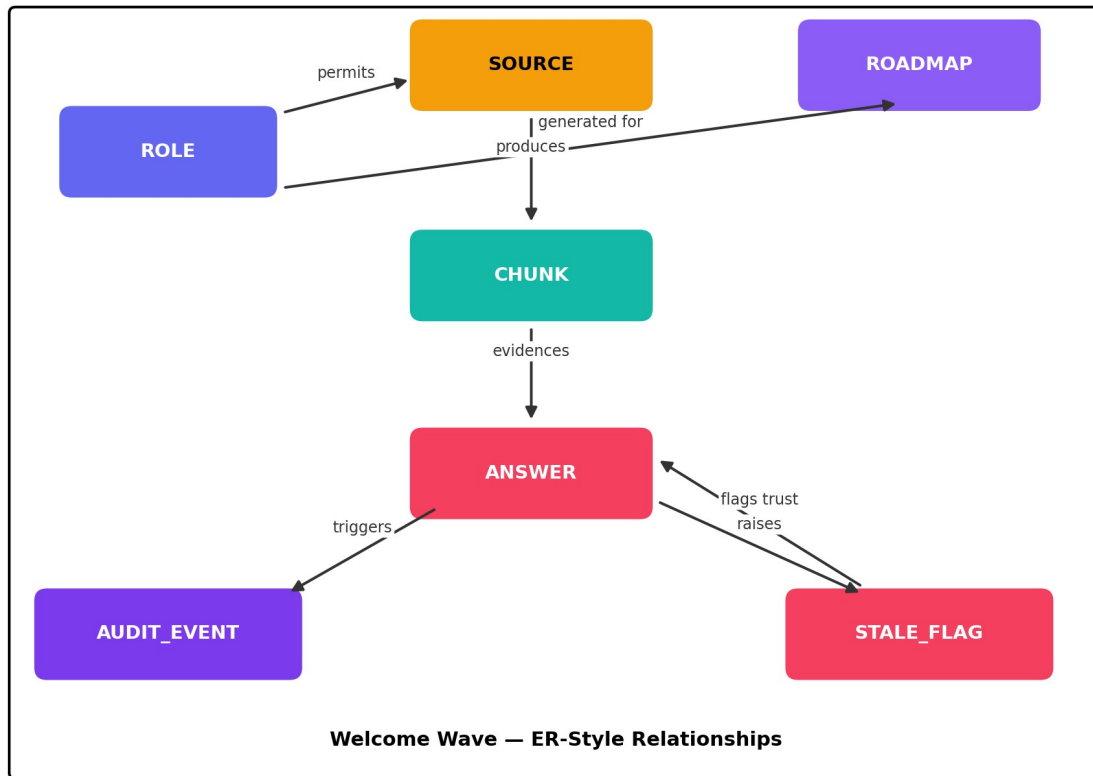
7. Database / Data Design

The retrieval path is largely stateless per-query, but the Trust Layer needs persistent state for sources, roles, verified answers, staleness flags, and audit history. The schema below covers that control-plane data store (relational, e.g. PostgreSQL).

7.1 Core Collections

Collection	Fields
sources	_id, name, type (document-upload github), role, permission, status (indexed error), connectedAt, lastIndexedAt
chunks	_id, sourceId, role, permission, text, embeddingId, createdAt
roles	_id, name, permittedSources (array), createdAt
answers	_id, question, role, status (ai_draft verified), text, evidenceChunkIds (array), verifiedBy, verifiedAt, sourceDependencyIds (array), createdAt
staleFlags	_id, answerId, sourceId, flaggedAt, resolvedAt
roadmaps	_id, role, overview, readFirst (array), firstTask, generatedAt
auditEvents	_id, actorId, action (CONNECT_SOURCE VERIFY_ANSWER FLAG_STALE ASK), metadata, timestamp

7.2 ER-Style Relationships



7.3 Indexes

- sources.name — unique
- chunks.sourceId + role
- answers.role + status
- answers.sourceDependencyIds (for staleness lookup)
- staleFlags.answerId
- auditEvents.actorId + timestamp

7.4 Data Lifecycle

Source connected → chunked + embedded → roadmap generated for its role → questions answered as drafts → experts verify into canonical → source updates → dependent answers flagged stale → re-verified → cycle continues.

- answers and auditEvents are append-only for traceability.
- Only synthetic/development knowledge is stored during the OJT build; no real employee PII flows through the system.

8. UX Requirements — Onboarding Console

8.1 Information Architecture

1. Landing / Role Select
2. Roadmap View
3. Ask & Evidence View
4. Verify Queue (domain experts)
5. Re-Verification Queue (staleness backlog)
6. Manager Dashboard
7. Source Connect (admin)

8.2 Primary User Flow

Select/assume a role → land on the auto-generated Roadmap → ask a question in the Ask view → see the grounded, evidence-cited, labeled answer → (if a domain expert) review the Verify Queue and confirm answers → manager checks the Dashboard for gaps and the re-verification backlog.

8.3 Key Screen Requirements

Roadmap View — role-specific overview, “read this first” list, suggested first task; default landing view, never the chat.

Ask & Evidence View — question input, grounded answer, trust badge (Verified vs AI draft), clickable evidence citations.

Verify Queue — list of pending AI drafts for the expert’s domain; verify or edit-then-verify actions.

Re-Verification Queue — list of previously verified answers now flagged stale, with the source change that triggered it.

Manager Dashboard — top recurring/unanswered questions per role, verified-coverage percentage, re-verification backlog size.

Source Connect — admin screen to connect a source (read-only), scope include/exclude, and assign a role.

8.4 States

Every async screen (Ask, Verify Queue, Dashboard) must define loading, empty, success, and error states, since all three depend on live retrieval or verification activity.

8.5 Visual Design Direction

Use a distinct accent color per functional area (violet for the roadmap/proactive layer, teal for ask/retrieval, amber for verification/trust, coral for staleness/errors) — consistent between architecture docs and the live console.

8.6 Accessibility

Full keyboard navigation across Roadmap, Ask, and Verify Queue; semantic labels on all interactive elements; adequate color contrast; responsive layout down to a single-column view; never rely on color alone to indicate trust state — pair badge color with icon and text.

9. Security Design

9.1 Authentication

Users authenticate via a token issued at login; the token carries the user's assigned role.

9.2 Authorization

Every retrieval and roadmap request is filtered by the role encoded in the token; a role cannot access content outside its `permittedSources` even if it knows a source name.

9.3 Transport Security

All traffic runs over TLS in any non-local deployment. Secrets (API keys, signing keys) are stored outside source code, via environment/secret management.

9.4 Ingestion-Level Security

Every connector is read-only by design — no write/update/delete method exists in the `IngestionConnector` interface. Admin-defined include/exclude scoping is enforced before any content is embedded.

9.5 Multi-Source Aggregation Security

A compromised or misbehaving source connector cannot affect another source's index; chunking and role-tagging are isolated per source registration.

9.6 Observability & Audit

All verification and staleness events are recorded in `auditEvents`. Metrics and logs never include raw document content — only identifiers and counts.

9.7 Resilience Against Adversarial Input

- Malformed ingestion files
- Oversized uploads
- Rapid connect/disconnect cycles
- Concurrent question floods against a single role

9.8 Important Product Rule

Welcome Wave is a knowledge-curation and workflow layer, not an autonomous decision-maker. It never writes back to a connected source, never auto-promotes a draft to verified without an explicit expert action, and never serves a role content outside its explicit permission scope under any failure condition.

10. Testing Strategy

10.1 Test Levels

1. Unit — chunking, role-filtering, verification logic, staleness dependency resolution
2. Integration — ingestion to retrieval to answer round trip
3. Trust Conformance — the internal verification suite (Section 5.7)
4. Resilience — malformed input, concurrency
5. Performance — retrieval and generation latency under load
6. UI — console components (Roadmap, Ask, Verify Queue, Dashboard)
7. End-to-End — full demo script across 3+ configured roles

10.2 Sample Test Cases

ID	Scenario	Expected Result	Priority
TEST-001	New role asks a question with no verified answer yet	Grounded AI draft returned, labeled unverified	High
TEST-002	Domain expert verifies a draft	Answer becomes canonical, trusted, served first afterward	High
TEST-003	Same question asked again after verification	Verified answer served instantly, no new LLM call	High
TEST-004	Role A asks about Role B's exclusive content	No result / access denied, no leakage	High
TEST-005	Source document is updated	All dependent verified answers flagged "needs re-verification"	High
TEST-006	Unrelated source is updated	Unrelated verified answers remain trusted, unaffected	High
TEST-007	Malformed document uploaded	Typed ingestion error, service stays up	Medium
TEST-008	Concurrent question flood on one role	Server degrades gracefully, no data corruption	Medium
TEST-009	Roadmap requested for a newly configured role	Roadmap generated correctly from that role's tagged content	High

ID	Scenario	Expected Result	Priority
TEST-010	Manager dashboard queried under load	Correct gap/coverage numbers, no stale cache shown	Medium

10.3 Trust Conformance Suite

See Section 5.7. The suite is run on every pull request touching retrieval, verification, or staleness logic.

10.4 Performance Benchmarking

- Baseline retrieval + generation latency per question
- Roadmap generation time on hire creation
- Staleness-flagging latency after a source update

10.5 Resilience / Adversarial Test Matrix

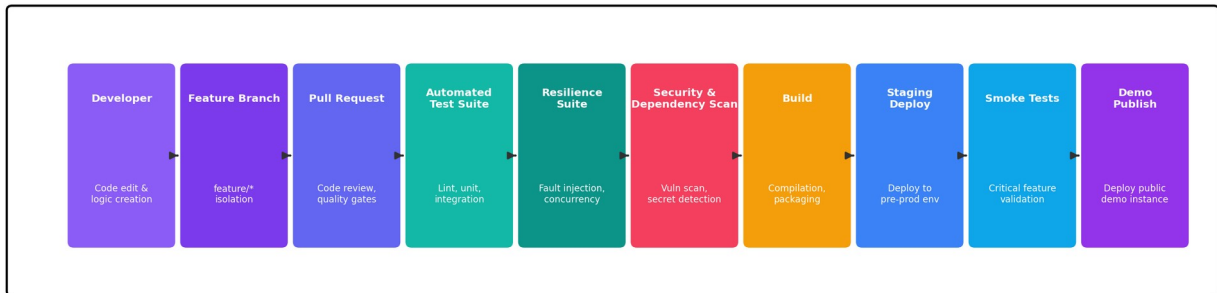
Fault Injected	Expected System Behavior
Malformed document upload	Rejected with a typed ingestion error, service stays up
Oversized file upload	Rejected before reaching the chunker
Concurrent verify actions on the same draft	Only one canonical answer results, no duplicate/lost update
Rapid source connect/disconnect	No orphaned chunks left in the index
Question flood on a single role	Rate limiting/backpressure engages, other roles unaffected

10.6 Definition of Done for a Feature

1. Unit and integration tests pass
2. It respects role/permission scoping where applicable
3. Relevant metrics are emitted and visible on the dashboard
4. It has been manually verified inside the live console across at least 2 roles

11. CI/CD

11.1 Pipeline



11.2 Branching Strategy

- **main** — stable, always demoable; every merge here is a candidate for the demo instance
- **develop** — integration branch for in-progress work across the two-person team
- **feature/*** — one branch per feature/task (e.g. feature/staleness-detection, feature/role-scoping)

11.3 Pull Request Requirements

Every PR must contain:

- Problem — what gap or bug this addresses
- Solution — the approach taken
- Testing — which test levels were run (unit / integration / conformance / resilience)
- Screenshots or logs — for console UI changes or new metrics
- Risks — anything that could affect role scoping, trust labeling, or the read-only guarantee

11.4 Quality Gates

A PR cannot merge when:

- Any required test tier fails (lint, unit, integration, conformance)
- A change allows cross-role content leakage
- A secret is detected in the diff
- A change breaks the “read-only ingestion” guarantee without an explicit, documented reason

11.5 Release Process

1. Merge to main triggers the full pipeline, including the trust conformance suite and resilience tests
2. On success, the demo instance is redeployed
3. A tagged release includes updated `architecture.md` and `api-reference.md`
4. Post-release smoke test verifies the demo instance is reachable and answers correctly across roles

11.6 Rollback

Every release is a versioned, reversible deployment; rollback means redeploying the previous tag. Verified answers and staleness flags are never destructively migrated during a rollback.

12. Observability

12.1 Logs

Every log line includes, where applicable: Request ID, Role, Source ID, Operation type, and Error code (if any). Raw document content is never logged in full — only identifiers and sizes, to avoid leaking sensitive content into logs.

12.2 Metrics

Metric	Type	Description
welcome_wave_questions_total	Counter	Total questions asked, labelled by role and status (verified/draft)
welcome_wave_answer_latency_seconds	Histogram	End-to-end latency per question (p50, p95, p99)
welcome_wave_verifications_total	Counter	Total verify actions, labelled by role
welcome_wave_stale_flags_total	Counter	Total staleness flags raised, labelled by source
welcome_wave_active_sources	Gauge	Currently connected knowledge sources
welcome_wave_roadmap_generations_total	Counter	Total roadmaps generated, labelled by role

12.3 Tracing

Trace path: Employee → Retrieval (role filter, search, generation) → Verification check → Response. Each span carries the request ID so a single question can be followed end-to-end, including through the role-scoping and trust-labeling steps.

12.4 Dashboards (Onboarding Console)

- Latency charts per role
- Verified-coverage over time
- Re-verification backlog size
- Questions-per-role volume

12.5 Alerts

- Elevated error rate on ingestion or retrieval

- A source stuck in “error” status
- Abnormal answer latency p99
- Re-verification backlog growing without being cleared

12.6 Post-Resilience-Test Review

After each resilience test run, observability data from that run is reviewed as part of the hardening phase (Week 11) to confirm the system degraded gracefully rather than failing silently.

13. Deployment Architecture

13.1 Environments

Development → Testing → Staging → Production/Demo

13.2 Suggested Deployment

React console served as a static web app; retrieval, verification, and roadmap services run as independent containers; PostgreSQL and the vector index in managed/local stores; Prometheus scrapes `/metrics` from each service.

13.3 Publishing Targets

- A demo instance deployed to a public URL for evaluators
- Architecture documentation and demo video published alongside the Git repository

13.4 Rollback

Versioned container images/releases; every deployment reversible by redeploying the previous tag.

13.5 Configuration Management

Source connector credentials, embedding/LLM API keys, and role configuration are environment-driven, never hard-coded.

13.6 Scaling Considerations

Start as modular services (ingestion, retrieval, verification, console) rather than a full microservices mesh. Introduce a queue between ingestion and the chunker only if source volume requires decoupling — not by default.

14. Glossary

Term	Definition
Welcome Wave	The AI onboarding workflow platform built in this project
Verified Answer / Canonical	An answer a domain expert has approved; trusted and served first
AI Draft	An unverified, freshly generated answer, clearly labeled as such
Staleness Flag	An automatic warning that a verified answer's source has changed and needs re-checking
Role Scoping	Restricting what content and roadmap a user can access based on their assigned role
Ingestion Connector	A pluggable module that reads a knowledge source (read-only) into the system
Roadmap	The auto-generated, role-specific onboarding path shown to a new hire
Verification Workflow	The process by which a domain expert turns an AI draft into a canonical answer
Trust Conformance Suite	The internal test suite validating role-scoping, verification priority, and staleness correctness